

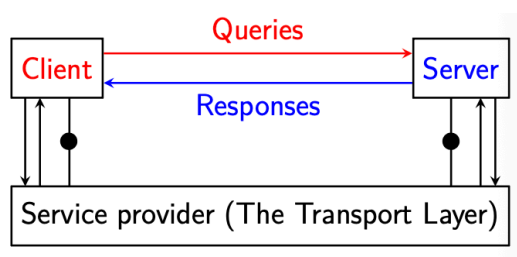
22 - 10

transport layer

Assumiamo che il network layer offra una consegna di SDU senza connessione e non affidabile: un servizio datagram, non basato su circuito.

le richieste del layer di applicazione sono che i dati arrivino **correttamente**, che il ricevitore possa **richiederli** nuovamente e assicurarsi che i **duplicati** vengano rilevati.

modello client- server



un host deve maneggiare numerose connessioni tra host e client nello stesso momento: multiplexing. il layer di trasporto deve poter distinguere il flow dei dati che appartiene a 2 diverse coppie client-server.

segment: l'unità di dati con cui lavora il layer di trasporto.

mentre le unità di network/layer sono i packets.

connectionless service

fornisce un servizio di trasporto dati non affidabile ma con error detection e il multiplexing dei due endpoint.



normalmente è senza stati, ciò significa che ogni segmento viaggia in modo indipendente e gli endpoint del layer di trasporto non hanno variabili di stato

usato in applicazioni a cui non interessa molto la perdita di dati (applicazioni interattive) e hanno bisogno che i pacchetti persi vengano ritrasmessi (gaming, streaming).

Trasport layer Headers

- per garantire l'error detection il layer di trasporto aggiunge un header con la checksum del segmento.
- sono presenti anche 2 porte; una del client e una del serve, quando il layer di trasporto riceve un segmento lo invia al corretto mittente in base al port number.

servizio di trasporto orientato a connessioni

il servizio crea una connessione tra 2 entità, la connessione ha uno stato quindi alcune variabili interne sono usate per controllare l'evoluzione della connessione.

il setup della connessione è facile come lo scambio di 2 pacchetti, considerando che la connessione non è affidabile.



devo poter distinguere due richieste di connessione duplicate da due diverse

- si può aggiungere un identificativo unico per ogni segmento. se riceviamo 2 segmenti con lo stesso identificativo droppiamo l'ultimo
- questo forza il server a mantenere una coda di tutti gli identificativi ricevuti

maximum segment lifetime MSL

il massimo tempo stimato che un pacchetto può rimanere nella rete. Assumiamo che $MSL < 120s$

- abbiamo bisogno di una variabile che aumenta con il tempo
- ogni entità del layer di trasporto ha un transport clock: un contatore che è incrementato col tempo e per ogni nuova connessione, non deve essere sincronizzato tra gli endpoint della connessione

initial sequence number ISN

ogni connessione ha un numero di sequenza iniziale, questo numero è il risultato di una funzione del timer.

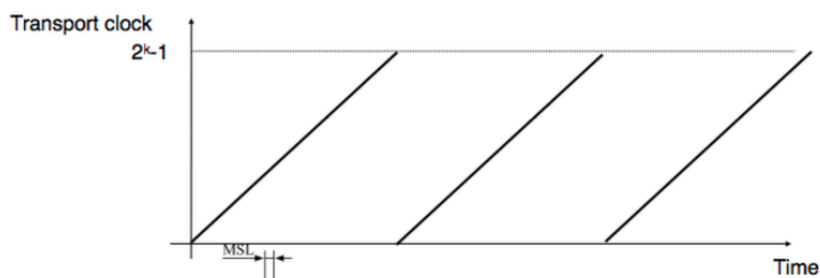


consideriamo che ISN è il contatore

il server rimanda il numero di sequenza nel messaggio CA quando il client riceve il messaggio e controlla se il numero di sequenza corrisponde.



l'ISN deve essere messo sull'header quindi ha una dimensione limitata



il wrap up timer del contatore è molto più largo del MSL

t_0 . nel caso in cui un client apra una connessione con $ISN = X$

t_1 . la connessione termina

t_2 . il client apre una nuova connessione con $ISN = X + \Delta$ verso lo stesso server. Ma risulta che $(X + \Delta) \% n = X$

t_3 . il client riceve il CA che riconosce X , ma non può sapere se si riferisca alla nuova o alla vecchia connessione

- possiamo assumere che il wrap dell'ISN sia molto grande comparato al MSL, allora la CA si riferisce alla nuova connessione.

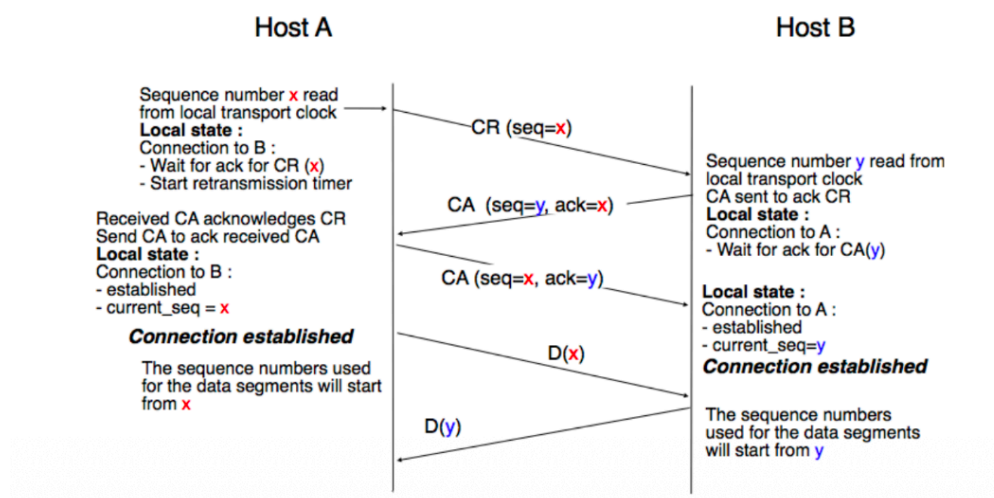


abbiamo bisogno che l'ISN wrappi in un intervallo di tempo molto superiore al MSL

two way communication

in generale vogliamo una connessione bidirezionale quindi abbiamo bisogno di 2 sequence number: una per ogni direzione di connessione.

three way handshake



1. il segmento CR contiene un numero di porta e un numero di sequenza che è estratto dal trasporto clock.
2. il server processa il segmento CR e crea lo stato per il tentativo di connessione, non sa ancora se è una connessione duplicata o una nuova connessione
3. rimanda una CA che contiene un numero di ACK per confermare che ha ricevuto il CR e un numero di sequenza il cui valore è estratto dal clock di trasporto.
4. il cliente riceve la CA il numero di ACK di questo segmento conferma se l'entità remota ha correttamente ricevuto il segmento CR

5. il client considera la connessione stabilita.
6. prima di inviare dati il cliente deve mandare un ACK di aver ricevuto il segmento CA
7. il server conferma la connessione stabilita se ha ricevuto l'ACK del segmento CA

trasporto di dati

il ricevitore ha un buffer perchè non c'è la dimensione di un frame come al layer2, finchè il buffer ha spazio il ricevitore può riempirlo di dati, solamente se le ACKs sono inviate subito.

`data.ind()` non è più bloccante, il ricevitore può ricevere finchè l'applicazione processa i dati.



il buffer può però essere sovraccaricato di dati, anche perchè le ACK sono mandate immediatamente

window Advertising

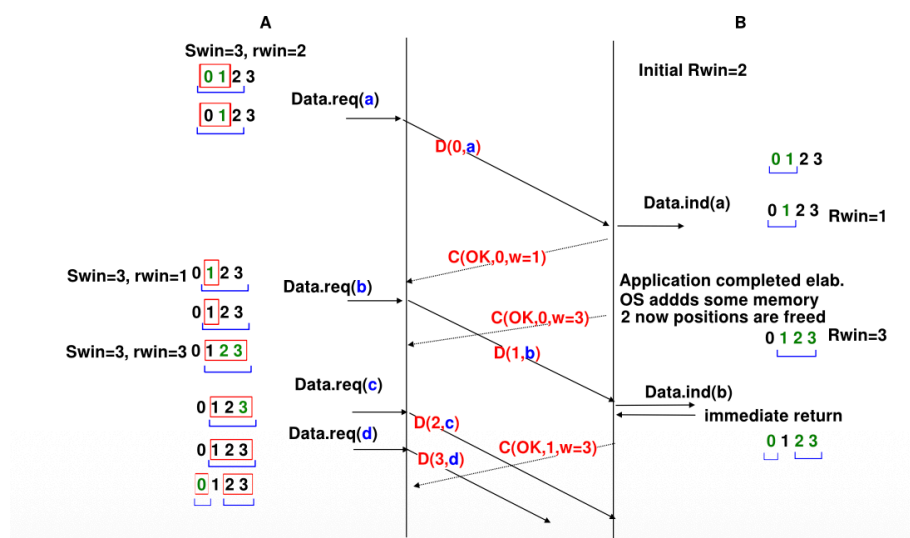
Per risolvere questo problema il protocollo di trasporto permette al ricevitore di pubblicare la dimensione corrente della **finestra di ricezione** in tutti gli ACK. Essa è legata alla dimensione del buffer.

swin: è la dimensione della finestra di invio.

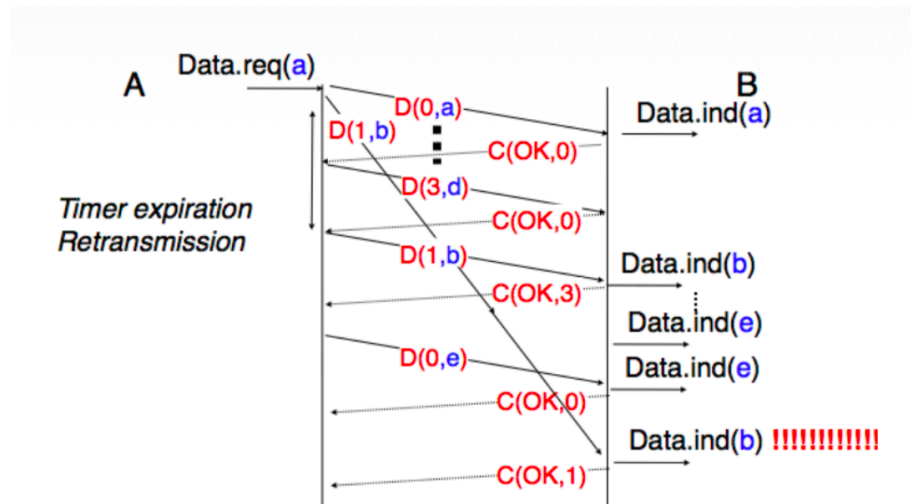
rwin: è la dimensione della finestra di ricezione



il numero di pacchetti non ACK deve essere sempre $< \min(swin, rwin)$



la finestra può anche essere nulla, quando succede rimanda periodicamente dati vecchi per dare al ricevitore l'opportunità di mandare un ACK, in caso uno fosse stato perso.



in questo esempio A manda il segmento 1 che però arriva dopo un intero wrap dell'ISN, visto che coincideva con il numero del nuovo ISN



B ha accettato il segment nonostante fosse vecchio corrompendo il file. Il sequence number non deve mai ripetersi all'interno della MLS

questo implica una restrizione sulla massimo throughput:

- assumiamo che la rete sia molto veloce: 30 Gb/s e i numeri di sequenza sono 32bit, e ogni numero di sequenza si riferisca a un byte:
- $2^{32} \simeq 4 * 10^9$ quindi il numero di sequenza si wrappa ogni 4 GB
- questo implica $\frac{2^{32} * 8}{30 * 10^9} \simeq 1s \ll 120s = MLS!!$

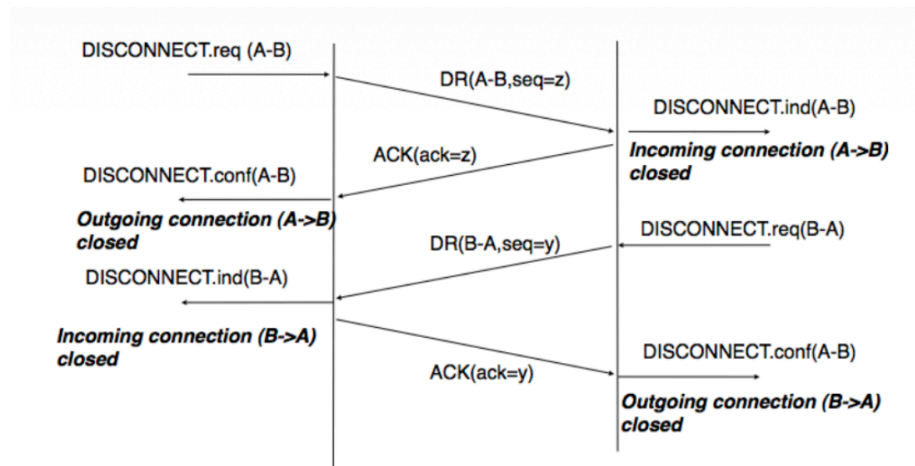
*capacity * delay*

quando la capacità è così alta spesso i pacchetti viaggiano su reti locali, quindi il delay reale è basso.

release

quando il trasferimento dei pacchetti è terminato la connessione va chiusa. Entrambi gli endpoint devono essere consapevoli che la connessione è stata chiusa.

graceful release



abrupt release

